

CITY-WIDE ANPR INTELLIGENCE PLATFORM

Comprehensive Technical Architecture, Implementation Methodology & Technology Stack Documentation

Project Domain: Smart City Traffic AI & Homeland Security

Target Accuracy: > 90% OCR Recognition Rate

Dataset Source: Kaggle Car Number Plate Video Dataset

System Core: Real-time Multi-Camera Distributed Pipeline

1. Executive Summary & Problem Context

Modern urban metropolises deploy vast networks of CCTV and Automatic Number Plate Recognition (ANPR) cameras. However, legacy systems process feeds in **isolated silos**, performing disconnected plate detections without linking spatial and temporal relationships. This lack of centralized intelligence prevents law enforcement from reconstructing suspect travel trajectories and limits city traffic planners from extracting macro-level movement dynamics.

This project establishes a unified, end-to-end **City-Wide ANPR Intelligence & GIS Analytics Platform** that processes concurrent CCTV feeds in real time, delivers high-accuracy deep learning OCR across challenging environmental conditions (poor lighting, weather, motion blur, steep angles), reconstructs cross-camera vehicle trajectories, and visualizes macro traffic dynamics (Origin-Destination matrices, congestion bottlenecks, and GIS heatmaps).

2. Implemented Core System Capabilities

A. High-Accuracy Deep Learning ANPR & OCR Engine

- **Vehicle Detection:** YOLOv8 deep learning detector identifying Car, Motorcycle, Bus, and Truck classes with vehicle color classification.
- **License Plate Localization:** Fine-tuned YOLOv8 License Plate bounding box model trained on real-world Kaggle CCTV video footage.
- **Image Preprocessing & Normalization:** Contrast Limited Adaptive Histogram Equalization (CLAHE), Bilateral filtering for motion deblurring, and aspect ratio rectification.
- **Deep OCR & Positional Grammar Correction:** Deep Learning OCR (EasyOCR / CRNN) coupled with Indian HSRP syntax normalization (State codes, district codes, series rules, number disambiguation e.g., O/0, I/1, Z/2, B/8).
- **Multi-Frame Temporal Voting:** Aggregates plate readings across consecutive video frames to produce a single, high-confidence consensus plate, eliminating single-frame false positives.

B. Multi-Camera Real-Time Signal Processing Engine

- **Concurrent Stream Architecture:** Multithreaded video processor handling simultaneous camera feeds across city junctions (Connaught Place, AIIMS Flyover, ITO Junction, DND Toll, etc.).
- **Real-time Ingestion & WebSocket Broadcast:** Sub-second event streaming pushing detection metadata to central FastAPI server and live dashboard clients.

C. Single Plate Spatial-Temporal Trajectory Reconstruction

- **Chronological Journey Mapping:** Queries historical camera sightings for any queried license plate across time and space.
- **Geodesic Distance & Speed Computation:** Uses WGS-84 geodesic algorithms (Geopy/Haversine) to compute hop-by-hop travel times, inter-camera distances, and average speeds (km/h).
- **Cloned Plate / Route Anomaly Detection:** Flags physically impossible travel speeds between distant cameras, alerting operators to cloned plates.

D. Macro Traffic Analytics & GIS Dashboard

- **Origin-Destination (OD) Matrix:** Aggregates vehicle movement patterns between city entry and exit points.
- **Congestion & Level of Service (LoS):** Real-time congestion index calculation based on vehicle throughput and sector speeds.
- **Geospatial Density Heatmaps:** Visual heatmaps overlaid on interactive Leaflet GIS city maps with temporal filtering.

E. Real-Time Alert & Hotlist Security Engine

- **Automated Hotlist Matching:** Real-time matching against Stolen, Wanted, Revoked, and Suspicious vehicle registries.
- **Speed Limit Enforcement:** Instant alerts triggered when vehicles exceed camera zone speed thresholds.
- **Sound & Push Notifications:** Real-time visual and audio alerts on the monitoring terminal.

3. Technical Architecture & End-to-End Workflow

Stage	Component / Engine	Implementation Details & Methodology
1. Ingestion	Multi-Camera Runner & Video Processor	Simultaneous decoding of video feeds (RTSP/CCTV/MP4). Frames sampled and batched to inference queue.
2. Detection	YOLOv8 Detection Pipeline	YOLOv8 detects vehicle bounding boxes; plate detector isolates license plate ROI with sub-pixel precision.
3. Enhancement	Adaptive Computer Vision Preprocessor	CLAHE contrast normalization, bilateral noise suppression, and morphological skew correction.
4. Recognition	Deep OCR + HSRP Regex Engine	Character recognition powered by PyTorch EasyOCR deep neural network + Indian state/series syntactic validator.
5. Temporal Voting	Multi-Frame Plate Aggregator	Sliding window voting resolves character-level character consensus, eliminating flickering & OCR jitter.
6. Backend & GIS	FastAPI REST & WebSocket Hub	Detections ingested to SQLite/PostgreSQL. Trajectories, speeds, and hotlist alerts computed in real time.
7. Command UI	React + Vite + Leaflet GIS Dashboard	Real-time CCTV grid, interactive map path reconstruction, traffic heatmaps, and alert center.

4. Comprehensive Technology Stack

Layer	Technology / Framework	Version	Role & Responsibility
AI / Computer Vision	Ultralytics YOLOv8 OpenCV EasyOCR / PyTorch scikit-image	v8.4.x v5.0.x v1.7.x v0.26.x	Vehicle detection, plate localization, image enhancement, neural OCR, and motion deblurring.
Dataset & Training	KaggleHub API PyTorch Torchvision	v1.0.x v2.13.x	Real car plate video dataset downloading, frame labeling, model fine-tuning and validation.
Backend API & WebSockets	FastAPI Uvicorn WebSockets Pydantic	v0.141.x v0.52.x v17.1 v2.13.x	Asynchronous microservices, sub-second event ingestion, real-time live push broadcasting, and request schemas.
Database & ORM	SQLAlchemy ORM SQLite / PostgreSQL	v2.0.x v3.x	Persistent relational storage for camera registry, detection events, trajectory logs, and hotlists.
Spatial Analytics	Geopy Pandas / Polars NumPy / SciPy	v2.5.x v3.0.x v2.5.x	WGS-84 geodesic distance, vehicle speed calculation, OD matrix aggregation, and density analytics.
Frontend Framework	React 19 Vite 8 TailwindCSS 4	v19.2.x v8.2.x v4.3.x	High-performance reactive user interface, multi-tile surveillance grid, dark-mode glassmorphic theme.

GIS & Visualization	Leaflet GIS Recharts Lucide React	v1.9.x v3.10.x v1.37.x	Interactive map trajectory routing, traffic heatmaps, congestion charts, and monitoring icons.
Package & Environment	uv (Astral) Python 3.12	v0.6.x v3.12	Blazing fast virtual environment, deterministic lockfile management, and cross-platform execution.

5. API Architecture & Key Endpoints

Method & Route	Payload / Parameters	Description & Output
POST /api/v1/detections/ingest	camera_id, plate_number, confidence, vehicle_type, vehicle_color, speed	Ingests real-time detection event, checks hotlist, and broadcasts to WebSocket clients.
GET /api/v1/trajectories/{plate}	plate_number (path)	Reconstructs chronological journey path, hops, inter-camera speeds, and cloned plate anomaly flags.
GET /api/v1/analytics/summary	hours (query, default 24)	Returns city-wide traffic volume, active alert count, average network speed, and vehicle distribution.
GET /api/v1/analytics/heatmap	None	Returns spatial coordinates with density weights for dynamic Leaflet GIS heatmap layers.
GET /api/v1/analytics/od-matrix	None	Origin-Destination traffic flow matrix showing vehicle travel counts between camera nodes.
GET /api/v1/alerts	limit, unresolved_only	Lists active security alerts, hotlist matches, speed limit violations, and cloned plate alerts.
WS /ws/live	WebSocket connection	Full-duplex real-time stream broadcasting detection events, alerts, and camera telemetry.

6. Deployment & Verification Quickstart

Step 1: Start Backend Server

uv run python run_backend.py
FastAPI backend starts at http://127.0.0.1:8000 with Swagger UI at /docs.

Step 2: Start Frontend Command Center

cd frontend && npm run dev
React + Vite dashboard starts at http://localhost:5173.

Step 3: Train & Validate Model on Kaggle Dataset

uv run python ai_pipeline/train_plate_detector.py
Fine-tunes YOLO plate detector on real video frames and computes mAP@50 metrics.

Step 4: Run Multi-Camera Real-Time Pipeline

uv run python ai_pipeline/multi_camera_runner.py
Streams real CCTV video feeds concurrently across all camera nodes and posts live detections.

Repository & Source Code: <https://github.com/maviyashaikh25/sih>